

Technical Blueprint — AEL pipeline agent

Version 0.4 (draft) · 24-09-2026 · Team: Alberto Barrionuevo, Pantelis Paraskevopoulos, Jasper Naalden, Nasjaat Potmis

Summary. This blueprint explains how the AEL pipeline agent works: six roles turn a raw source into a published handbook page, hand fixed files to each other, and reach the outside world only through one Gateway. It follows the chapters of blueprint v0.3, updated to AIBS/PRD version 4. AIBS = research proposal, PRD = product requirements document; a bare number refers to this blueprint.

Sections 1.2.1, 1.2.2, 1.4, 1.6 and 1.7 each answer one of the five lecturer feedback questions of week 2; the answered question is named at the start of the section.

Version	Date	Reason
0.3	24-09-2026	Split into its own document; added 1.2.1, 1.2.2 and 1.7.
0.4	24-09-2026	Updated to AIBS/PRD v4: search strategies, source quality criteria, lookup by id, PRD 2.3.6, non-coder operation and manual fallback.

1.1 Two altitudes

Figure 1 shows who does what. Figure 2 shows what follows what, and the file each role hands on.

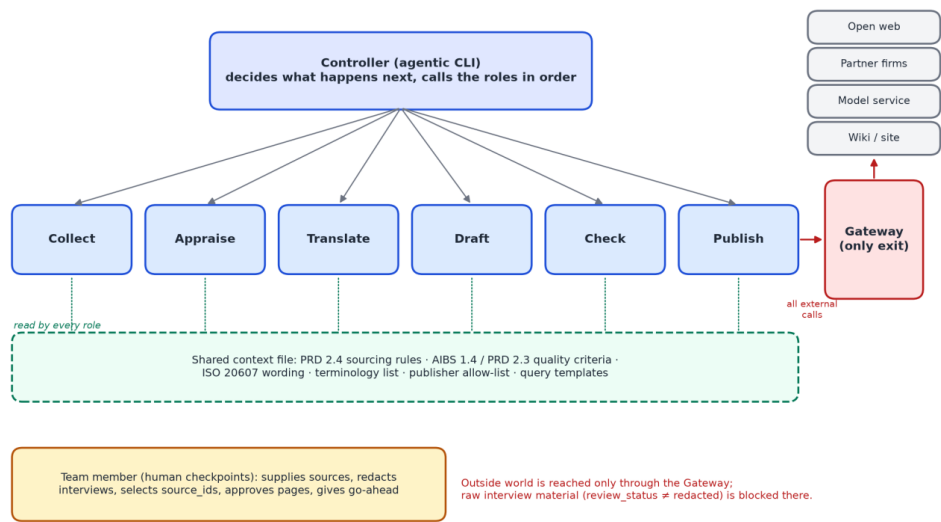


Figure 1 — Roles, control and shared context.

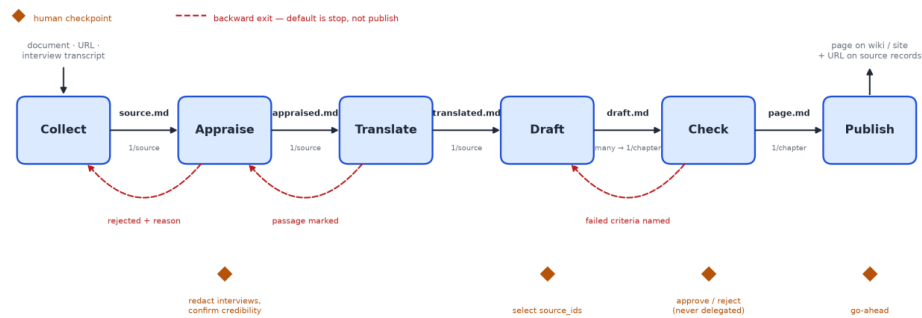


Figure 2 — The production line, with backward exits and human checkpoints.

1.2 The six decisions

- **Parts.** Six roles, each with a one-sentence job: Collect, Appraise, Translate, Draft, Check, Publish.
- **Control.** One controller (the agentic CLI) decides what happens next and calls the roles in order.
- **Contracts.** Roles hand over fixed files: source.md → appraised.md → translated.md → draft.md → page.md.
- **Shared context.** One team-written file holds all rules, criteria, terminology, the publisher allow-list and query templates. Every role reads it (new in v0.4).

- **The line.** Only the Gateway talks to the outside world. It blocks unredacted interview material (1.4).
- **Being wrong.** Check has three layers: code tests yes/no criteria, a model flags wording, a person approves. Only the person can let a page through.

1.2.1 What each role takes in and hands on

Answers lecturer feedback (week 2), question 4: what does each role take in and hand on?

Role	Takes in	Hands on	If it cannot
Collect	A document, URL or transcript, or a search request for one sub-question.	One source.md per source.	Nothing is created; the supplier is told.
Appraise	One source.md.	One appraised.md (fields in 1.3).	Too old, wrong type or language: rejected with a reason, back to Collect. Unclear publisher or metadata: flagged for a person.
Translate	One appraised.md.	One translated.md in English, also for English sources.	Back to Appraise with the passage marked.
Draft	Several translated.md files, chosen by a person.	One draft.md per chapter; every claim carries its source_id.	Reports which section has no source.
Check	One draft.md.	One approved page.md.	Back to Draft with the failed criteria named.
Publish	One approved page.md.	The page on the wiki or site; URL recorded.	Reports the failure; a person can publish by hand.

Draft is the one many-to-one step. The default on any failure is to stop, not to publish. Collect searches with stored query templates: one for literature (sub-questions 1–4 and 6) and one for case studies (sub-question 5), each in English and Dutch. They are editable instructions, not code. Interview transcripts are never searched for; a team member supplies them.

1.2.2 Who performs each part

Answers lecturer feedback (week 2), question 5: person, code or model per part, and why a model rather than a script?

Part	Person	Code	Model	Why a model
Collect	Supplies source or picks sub-question	Runs query, fetches, assigns source_id	—	Not used
Appraise	Redacts interviews; confirms credibility and tags	Checks age, type, language, allow-list, metadata	Writes summary, findings, opportunities, risks, tags	Deciding what is a finding needs judgement
Translate	Verifies key terms	Reads language; always writes file	Translates to English	Knows which terms stay Dutch
Draft	Selects source_ids	Opens files by id; builds template	Writes chapter text	Combines sources into one argument
Check	Approves or rejects (never delegated)	Tests source_ids, sections, APA, no identifying details	Flags unclear wording (ISO 20607)	Can flag, never approve
Publish	Gives go-ahead	Writes page, records URL	—	Not used

Every model step is followed by a human checkpoint, because a model's mistakes look plausible. Mechanical checks are code, so they are free to run and cannot be argued out of a "no".

1.3 What passes between the parts: appraised.md

Field	Type	Meaning
source_id	string	Assigned at Collect; every later file refers to it.
source_type	enum	academic professional government vendor interview.
language	enum	NL EN; tells Translate whether to translate.
publication_date	date	Enforces the 4-year rule (PRD 2.4).
publisher (new)	string	Checked against the allow-list; unlisted = flagged.
summary, key_findings, opportunities, risks	string / list	Written by Appraise; each item traces to source_id.
subquestion_tags (new)	list<int>	AIBS sub-questions 1–6; confirmed by a person.
flags (new)	list<string>	Open issues for a person, e.g. missing metadata.

review_status	enum	pending redacted approved rejected.
---------------	------	---

1.4 The line nothing may cross

Answers lecturer feedback (week 2), question 2: the PRD requirement behind the redaction rule is PRD 2.4 "Handling of interview material", with the prohibited inputs in PRD 2.5 and the output quality in PRD 2.3.6.

Raw interview material never reaches a model service. It is stored as an ordinary source.md; what protects it is that it cannot move on until a person clears it:

1. Collect labels it source_type = interview, review_status = pending.
2. Every source bound for a model must pass the Gateway; no other role calls out.
3. The Gateway refuses any interview source that is not "redacted". The model never sees the raw text.
4. Only a team member can set "redacted", by removing names, employers and identifying details by hand.
5. After that, the Gateway lets the anonymised content through.

On the output side, Check rejects any page that identifies an interviewee without written consent (PRD 2.3.6). The rule holds because transcripts contain personal data, companies expect their material to stay private, and the chapter template requires anonymised field material.

1.5 Where the map stops

- **Evaluation.** Two checks: Appraise tests sources automatically; Check is the human gate for pages.
- **Monitoring.** Stop, not publish. Failures go back with a reason. If the model fails, a person makes the same file by hand (manual fallback).
- **Memory.** Carried: the shared context file. Looked up: each source's content. Never carried: raw interview text.
- **Retrieval.** No search index. A person picks source_ids and Draft opens those files, which keeps every claim traceable. We switch only if the pool gets too big to pick by hand or readers need live answers.
- **Open items.** The model service is not yet named in PRD 2.4; the time saving is untested (week-3 test); no owner after the project ends.

1.6 The trace to the PRD

Answers lecturer feedback (week 2), question 3: the promise cited as "1.1" in v0.2 is AIBS 1.1 in the research proposal, not a PRD section; it is a research finding about SMEs, so the PRD now carries it as the constraint "Model service behind the Gateway" (PRD 2.4), and the row below points there.

PRD promise	Lands in	Note
2.2 Workflow steps 1–6	The six roles	One role per step.
2.3.1–2.3.5 Output qualities	Draft and Check	Check is the approval gate.
2.3.6 Protect interviewees; 2.4 interview handling; 2.5 prohibited inputs	Gateway, Appraise, Check	Blocked going in, tested going out (1.4).
2.4 Sourcing rules; AIBS 1.5–1.6 search and quality	Collect, Appraise, shared context	Auto-reject, flag or tag.
2.4 Agentic CLI; non-coder operation	Controller, not a part	Roles are instructions plus file shapes (1.7).
2.4 Manual fallback	File contract	Any file can be made by hand.
2.4 Existing infrastructure	Publish	Uses the existing wiki or site.
2.4 Model service behind the Gateway	Gateway	One place to switch; service still open.
2.1.1 Benchmark and survey; 2.6 out of scope; 2.7 open questions	Outside the pipeline	Not parts.

1.7 Why an agentic CLI, and not a person or a script

Answers lecturer feedback (week 2), question 1: what an agentic CLI offers over a person or a conventional script.

Task	By hand	Script	Agentic CLI
Summarise	Slow; each person formats differently	Cannot judge what matters	Fills the same fields every time
Findings, opportunities, risks	Needs subject knowledge	Cannot tell opportunity from risk	Reads and classifies any format
Translate Dutch	Not everyone reads Dutch	Cannot keep key terms	Keeps terms in context

All three tasks need an understanding of language, and the team has no developer. With an agentic CLI the agent is instructions and file contracts, so a changed criterion means editing an instruction, not code. Judgement stays with people, and the time saving is still an expectation, not a measurement.